SpendWise — Master Instructions

Read this first: project brief, rules for Antigravity, features and security requirements

1. Project purpose

SpendWise is a student-focused personal finance web application. It helps users record income and expenses, set monthly budgets, understand spending patterns, and evaluate a planned purchase. It is a learning and tracking tool, not a bank, payment processor, or source of professional financial advice.

2. Required assessment stack

- Frontend: Next.js App Router + TypeScript + Tailwind CSS.
- Backend: Amazon API Gateway HTTP API + AWS Lambda (Node.js 22).
- Database: Amazon DynamoDB.
- Object storage: Amazon S3 for optional receipt uploads; keep the bucket private.
- Authentication: Amazon Cognito User Pools with JWT authorization.
- Infrastructure as code: AWS SAM / CloudFormation.
- Hosting: AWS Amplify Hosting for the Next.js frontend.
- EC2 is prohibited. Do not create or rely on EC2 instances.

3. User stories

- As a student, I can record an expense with a description, amount, category, date and payment method.
- As a student, I can record income separately from expenses.
- As a student, I can edit or delete my own transaction and see the dashboard update.
- As a student, I can set a monthly overall budget and optional category budgets.
- As a student, I can see the percentage of my budget used and warnings at 75%, 90% and 100%.
- As a student, I can filter transactions by date, category and type.
- As a student, I can compare this month's spending with the previous month.
- As a student, I can check how a planned purchase would affect my remaining budget.
- As a user, I cannot see another user's transactions, budgets or receipt files.

4. MVP versus optional features

MVP means the minimum viable product required for a credible demonstration. Complete the MVP before optional enhancements.

MVP

- Real sign-in and sign-out.
- Create, read, update and delete transactions persisted in DynamoDB.
- Monthly budget creation and progress calculations.
- Dashboard totals and category analytics derived from saved records.
- Secure user-level authorization in the backend.

- Production build, hosted website, README and test evidence.

Optional only if time remains

- S3 receipt upload via short-lived presigned URL.
- CSV export, additional chart types, dark mode, recurring transactions or advanced insights.

5. Suggested repository structure

```
spendwise/\n  frontend/\n    app/\n    components/\n    lib/api.ts\n    lib/auth.ts\n    types/\n    package.json\n  backend/
```

6. Development sequence

- First: design and implement the responsive frontend using isolated mock data.
- Second: build the AWS backend and infrastructure definition.
- Third: connect the UI to the actual API and authentication.
- Fourth: test calculations, validation and cross-user authorization.
- Fifth: deploy the backend, configure Amplify, test the public site and prepare submission.

7. Critical safety and security rules

Never put AWS secret keys in browser code. Never trust a userId sent by the client. Derive the user identity from verified authentication claims and enforce ownership in every protected backend operation. Keep S3 private. Do not claim an API is working unless it was tested.

8. Time and cost discipline

The deadline is 28 October 2026 at 12:00 noon. Keep the scope small. Reserve time for deployment and debugging. AWS resource usage can incur charges; review the resources created by SAM and clean up unneeded resources after the assessment only when you are sure they are no longer needed.

9. How to use these PDFs

- Send the Master Instructions PDF prompt first. It asks Antigravity to work in stages rather than generating the whole system at once.
- Then use the Frontend, Backend, Integration, QA and Deployment PDFs in order.
- After each stage, inspect the output, run the commands, and fix errors before continuing.

COPY THE FULL PROMPT BELOW INTO ANTIGRAVITY

Use this prompt as one message. Let Antigravity inspect the repository, make the requested changes, run checks, and report what it actually completed. Do not skip the checkpoints.

You are my senior software engineer, product designer, AWS serverless architect, and patient coding mentor. I am building SpendWise — Student Expense Tracker for the Cybernetics Software Pvt. Ltd. Junior Software Developer assessment. I am using Antigravity and I am a beginner with AWS. Explain commands and cloud-console steps clearly.

```
PROJECT GOAL
Build a polished, responsive, working student finance web application. Students can record income and expenses, manage monthl

MANDATORY STACK
- Next.js App Router and TypeScript for frontend.
- Tailwind CSS and Lucide React for UI.
- Amazon API Gateway HTTP API.
- AWS Lambda using Node.js 22 and JavaScript ES modules.
- Amazon DynamoDB for transactions and budgets.
- Amazon S3 for optional receipt uploads, with public access blocked.
- Amazon Cognito User Pools for authentication and JWT-based authorization.
- AWS SAM / CloudFormation for backend infrastructure as code.
- AWS Amplify Hosting for the frontend.
- Do not use EC2.

CORE FEATURES
1. Landing page and authenticated dashboard.
2. Create, list, view, edit, delete income and expense transactions.
3. Transaction fields: description, amount, type, category, transaction date, payment method, optional notes, createdAt and u
4. Categories: Food, Travel, Shopping, Education, Entertainment, Bills, Health, Income and Other.
5. Search and filter by date range, category and type.
6. Monthly overall budget and optional category budgets.
7. Budget progress and alerts at 75%, 90% and 100%.
8. Dashboard totals: total income, total expenses, budget remaining, recent transactions and category breakdown.
9. Analytics: category spending and current-versus-previous-month comparison.
10. “Can I Afford This?”: user enters a planned purchase amount; show projected remaining budget and a clear warning if it ex
11. Settings with INR (■) as default currency.
12. Optional receipt upload only after core features are stable.

SECURITY AND DATA INTEGRITY
- Require real authentication for personal data.
- Use verified Cognito claims to derive the authenticated user identity.
- Never accept a client-supplied userId as proof of ownership.
- Every read, create, update and delete must enforce user-level authorization in the backend.
- Keep S3 private and use short-lived presigned URLs for uploads if implemented.
- Do not expose credentials, secrets, private data or raw internal errors.
- Validate request bodies and dates, amounts, categories and transaction types.
- Avoid DynamoDB full-table scans for ordinary user queries.
- Do not create fake security, unauthenticated admin routes, or hardcoded passwords.
- Use environment variables for configuration and commit only an .env.example with placeholders.

DESIGN QUALITY
Create a professional, premium financial dashboard with a warm off-white background, deep navy text, violet/indigo accents, s

WORKING RULES
- First inspect the repository, package manager, existing files and current build status.
- Do not overwrite useful existing work without explaining why.
- Work in small phases: frontend, backend, integration, testing, deployment.
- Do not attempt every phase in a single giant change.
- After each phase, list files changed, commands run, results, problems and exact next steps.
- Run lint, type checks, tests and production build where available; fix errors before moving on.
- If a required AWS account permission, console action, environment variable or secret is missing, explain it clearly and wai
- Do not claim a feature, test, cloud resource or deployment succeeded unless it actually did.
- Never invent the deployed URL.
- Avoid adding dependencies unless needed.
- Do not run destructive commands or delete cloud resources without asking me first.

START NOW
1. Inspect the repository and report what exists.
2. Propose a short implementation plan and identify any blocking issues.
3. Begin only with the frontend UI and isolated mock data.
4. Do not implement the backend until I confirm the frontend stage is ready.
5. Keep the code clean and beginner-friendly, with clear setup instructions.
```

I will provide the detailed stage-specific PDF prompts after this master instruction. Treat those documents as the detailed specification for each phase.